

Project Report: EoMT Model Evaluation and Fine-Tuning

AGA

Student ID: -

Politecnico di Torino

May 27, 2026

Task 1: Comparative Analysis of Cityscapes-Trained vs. COCO-Trained EoMT Models

1. Task Objective

The primary objective of this task is to conduct a rigorous, quantitative comparison between two distinct versions of the Ensemble of Masked Transformers (EoMT) model. The first version was trained on the Cityscapes dataset, a benchmark focused on urban street scenes. The second version was pre-trained on the COCO (Common Objects in Context) dataset, a large-scale, general-purpose object detection and segmentation dataset.

The comparison evaluates the performance of both models across three fundamental computer vision tasks:

1. **Semantic Segmentation:** Assigning a class label to every pixel in an image (e.g., car, road, person).
2. **Instance Segmentation:** Detecting and segmenting individual object instances within an image.
3. **Panoptic Segmentation:** A unified task that combines both semantic and instance segmentation, providing a comprehensive scene understanding by labeling all pixels with a class and, for “thing” classes, a unique instance ID.

The analysis is performed on a common subset of classes shared between Cityscapes and COCO to ensure a fair and direct comparison of the models’ capabilities in a controlled urban environment context provided by the Cityscapes validation set.

2. Architecture & Code Logic

The comparative analysis is orchestrated by the primary script, `compare_eomt_cityscapes_vs_coco.py`. This script is responsible for loading the models, processing the validation data, executing inference, calculating performance metrics, and generating visualizations.

Model Loading and Configuration:

The `load_eomt_model` function is the entry point for instantiating the models. It dynamically constructs the EoMT architecture based on a provided YAML configuration file. A key architectural detail is the handling of the Vision Transformer (ViT) backbone’s positional embeddings. The function `maybe_fix_pos_embed_in_state_dict` intelligently resizes the positional embeddings from a pre-trained checkpoint to match the input image resolution required for the current task (512x1024 for Cityscapes). This is achieved via 2D bicubic interpolation and is critical for adapting a model pre-trained on one image size (e.g., from COCO) to another.

The two models being compared are:

1. **Cityscapes-Trained EoMT:** Loaded using a configuration pointing to a model trained directly on Cityscapes.
2. **COCO-Trained EoMT:** Loaded using a configuration from a COCO pre-training session. The script fetches the corresponding weights from the Hugging Face Hub, identified by the model name specified in the YAML file (e.g., `facebook/eomt-large-coco-panopt`).

Data Processing and Class Mapping:

To ensure a fair comparison, the evaluation is restricted to a set of eight classes that are present in both the COCO and Cityscapes datasets. This mapping is explicitly defined in the `COCO_CONFIG_TO_CITYSCAPES_TRAINID` dictionary within the script. This dictionary maps the contiguous category IDs from COCO to the corresponding `trainIds` used in Cityscapes. The shared classes include `person`, `bicycle`, `car`, `motorcycle`, `bus`, `train`, `truck`, and `traffic light`.

The script iterates through the Cityscapes `val` set using the `_cityscapes_pairs` generator. For each image, it performs the following steps:

1. Loads the input image and the ground truth segmentation labels.
2. Runs inference with both the Cityscapes-trained and COCO-trained models to get panoptic predictions.
3. The predictions from the COCO model, which are in the COCO class space, are mapped to the Cityscapes class space using the `_map_coco_pred_to_cityscapes_known` function.
4. Both sets of predictions (the remapped COCO prediction and the original Cityscapes prediction) are then filtered by the `_mask_cityscapes_pred_to_known` function to ensure only the shared, known classes are considered for metric calculation.

Metric Computation:

The script calculates metrics for all three segmentation tasks:

- **Semantic mIoU:** The `_update_inter_union` function accumulates the intersection and union values for each known class across all validation images. The final mean Intersection-over-Union (mIoU) is computed from these accumulated values.
- **Instance mAP:** The script leverages the `torchmetrics.detection.MeanAveragePrecision` utility to calculate the mean Average Precision (mAP) for instance segmentation.

- **Panoptic Quality (PQ):** The `torchmetrics.detection.PanopticQuality` utility is used to compute PQ and its components: Segmentation Quality (SQ) and Recognition Quality (RQ).

The final computed metrics are saved to `compare_outputs/metrics_sem_inst_panoptic.csv`, and a human-readable report is generated at `compare_outputs/summary_sem_inst_panoptic.txt`.

3. Methodological Rationale

The methodology was designed to isolate and measure the impact of the training dataset on the model’s performance in a specific target domain (urban scenes).

- **Why Compare on a Shared Class Subset?** COCO has 80 “thing” classes, while Cityscapes has 8 “thing” classes and 11 “stuff” classes. A direct comparison across all classes would be invalid. By evaluating only on the classes common to both, we can directly assess how well the features learned from a general-domain dataset (COCO) transfer to a specialized domain (Cityscapes) compared to a model trained directly on that specialized domain.
- **Why Panoptic Segmentation?** Panoptic segmentation provides the most holistic evaluation. It penalizes models for failures in semantic understanding (miscategorizing a pixel), instance detection (missing an object), and segmentation quality (inaccurate masks). Evaluating semantic mIoU, instance mAP, and Panoptic Quality separately allows for a granular diagnosis of each model’s strengths and weaknesses. For instance, a model could be good at classifying pixels correctly (high semantic score) but poor at distinguishing between adjacent instances of the same class (low instance/panoptic score).
- **Code Architecture:** The choice to use a modular script (`compare_eomt_cityscapes_vs_coco.py`) that programmatically loads models and runs a validation loop ensures reproducibility and consistency. Hard-coding the class mappings and evaluation logic within this script guarantees that both models are subjected to the exact same evaluation protocol, eliminating procedural variables from the comparison.

4. Results & Analysis

The quantitative results of the comparison are detailed in the attached files `metrics_sem_inst_panoptic.csv` and `summary_sem_inst_panoptic.txt`.

Metric	Cityscapes-Trained Model	COCO-Trained Model
Semantic mIoU	0.7676	0.4024
Instance mAP	0.4177	0.4299
Instance mAP@50	0.6029	0.6623
Panoptic Quality (PQ)	0.5326	0.4797
Segmentation Quality (SQ)	0.7802	0.7977
Recognition Quality (RQ)	0.6366	0.5877

Analysis of Results:

1. **Semantic Segmentation (mIoU):** The Cityscapes-trained model achieves a **semantic mIoU of 0.7676**, which is nearly double the **0.4024** achieved by the COCO-trained model. This significant gap indicates that for pixel-level classification within the urban domain, specialized training is far superior. The model trained on Cityscapes has learned the specific appearance and context of urban classes with much higher fidelity than the generalist COCO model. The `summary.txt` file further breaks this down, showing the COCO model struggles particularly with classes like `truck` (0.14 IoU) and `motorcycle` (0.22 IoU), suggesting it confuses them with other vehicle types.
2. **Instance Segmentation (mAP):** Interestingly, the COCO-trained model slightly outperforms the Cityscapes model in instance detection, with an **mAP of 0.4299** versus **0.4177**. The gap is more pronounced at the less strict IoU threshold of 0.50 (**mAP@50 of 0.6623 vs. 0.6029**). This suggests that training on the diverse and numerous instances in the COCO dataset endowed the model with a more robust general ability to delineate object boundaries, even if it is less accurate at classifying them.
3. **Panoptic Segmentation (PQ):** Panoptic Quality combines semantic and instance performance. The Cityscapes-trained model achieves a higher overall **PQ of 0.5326** compared to the COCO model’s **0.4797**. This is primarily driven by its superior **Recognition Quality (RQ) of 0.6366 vs. 0.5877**, which reflects its strong semantic performance. Although the COCO model had slightly better mask quality (**Segmentation Quality (SQ) of 0.7977 vs. 0.7802**), its frequent misclassifications heavily penalized its final panoptic score.

Conclusion: The results demonstrate a clear trade-off. Training on a specialized dataset like Cityscapes yields a model with superior semantic understanding and overall panoptic performance in that domain. However, pre-training on a large, diverse dataset like COCO can provide a better prior for the task of pure instance delineation. The Cityscapes-trained model is the better overall performer for comprehensive scene understanding in urban environments, but the COCO model’s strong instance segmentation capability highlights the value of large-scale pre-training for developing robust object feature extractors.

Task 2: Fine-Tuning the COCO-Pre-trained EoMT Model on Cityscapes

1. Task Objective

The objective of this task is to adapt the EoMT model, which was pre-trained on the general-purpose COCO dataset, to the specific domain of urban street scenes using the Cityscapes dataset. The goal of fine-tuning is to leverage the rich, general-purpose features learned from COCO’s diverse object classes and contexts and specialize them for the classes and environmental conditions present in Cityscapes.

This process aims to improve the model’s semantic and panoptic segmentation performance on the target domain, potentially creating a model that combines the robust instance detection capabilities of COCO pre-training with the domain-specific accuracy of Cityscapes training.

2. Architecture & Code Logic

The fine-tuning process is primarily configured through YAML files and implemented within the core training scripts of the `eomt` module. The architecture allows for flexible fine-tuning strategies by selectively unfreezing parts of the pre-trained model.

Configuration Files:

The fine-tuning experiments are defined in two main configuration files:

1. `eomt/configs/finetune/cityscapes/panoptic/coco_to_cityscapes_head_only.yaml`:
This configuration specifies a fine-tuning strategy where only the final prediction “head” of the model is trained. All layers of the ViT backbone and the transformer decoder remain frozen.
2. `eomt/configs/finetune/cityscapes/panoptic/coco_to_cityscapes_last4.yaml`:
This configuration implements a more aggressive strategy where, in addition to the prediction head, the last four residual blocks of the ViT backbone are also unfrozen and trained.

Both configurations inherit from the base COCO training configuration (`eomt_base_640.yaml`) and specify the path to the pre-trained COCO model checkpoint, which serves as the starting point for the fine-tuning process.

Code Implementation:

The logic for applying these strategies is implemented in `eomt/training/lightning_module.py`.

- The `MaskClassificationPanoptic` Lightning module accepts `finetune_strategy` and `finetune_unfreeze_last_blocks` as initialization arguments, which are passed down from the YAML configuration.
- Inside the `__init__` method, a call to `self._apply_finetune_strategy()` triggers the parameter freezing logic.
- The `_apply_finetune_strategy` method first freezes all parameters in the entire network (`self.network.requires_grad_(False)`).
- Based on the `finetune_strategy` string:

- If the strategy is "last_k_blocks" (as used by `coco_to_cityscapes_last4.yaml`), it proceeds to unfreeze the parameters of the last k blocks of the encoder backbone. The number k is specified by `finetune_unfreeze_last_blocks`.
- Regardless of the backbone strategy, the method always unfreezes the parameters of the prediction heads (`self.network.class_head` and `self.network.mask_head`). This ensures that the model can learn to map the backbone features to the new Cityscapes class labels.

The training process itself is launched via `eomt/main.py`, which parses the specified YAML config, instantiates the `MaskClassificationPanoptic` module with the correct fine-tuning settings, and starts the PyTorch Lightning Trainer.

3. Methodological Rationale

The choice of fine-tuning strategies represents a standard, principled approach to transfer learning, designed to balance knowledge retention with domain adaptation.

- **Why Start with a Pre-trained Model?** The COCO dataset contains millions of object instances across 80 categories. Training on this data allows the model’s ViT backbone to learn a powerful and generalized visual representation. These features, which encode fundamental concepts like shape, texture, and object composition, are highly transferable to other computer vision tasks. Starting with these pre-trained weights is significantly more efficient than training from scratch and typically leads to better performance.
- **Rationale for “Head-Only” Fine-Tuning (`head_only`):** This is the most conservative and computationally efficient strategy. The hypothesis is that the pre-trained backbone already functions as a strong, universal feature extractor. By freezing the backbone and only training the final classification and mask heads, we are simply learning a new mapping from the existing feature space to the Cityscapes class labels. This approach minimizes the risk of “catastrophic forgetting,” where the model loses its valuable general features by overwriting them during training on the new, smaller dataset.
- **Rationale for “Last k Blocks” Fine-Tuning (`last4`):** This strategy is a compromise between full fine-tuning and head-only tuning. In a ViT, earlier layers tend to learn low-level features (e.g., edges, colors), while deeper layers learn more abstract, high-level semantic features. The hypothesis here is that the high-level features are more dataset-specific. By unfreezing and training the last few blocks, we allow the model to adjust its high-level feature representation to better suit the nuances of the Cityscapes domain, while still keeping the foundational low-level features learned from COCO intact. This often provides a performance boost over head-only tuning without the high computational cost and risk of full fine-tuning.

By setting up these two experiments, the project is prepared to systematically evaluate which fine-tuning approach yields the best-performing model for panoptic segmentation in an urban environment.